

CSIC30216

AI-in-the-Loop in Software Project Cycle

AI 導入軟體專案流程

Week 5:
Prompt Engineering I -
Writing Prompts Well

Instructors:
Dennis Wang 王邦任
Yu-Chun (Grace) Yen 顏羽君

Date: 2026-10-05

Announcements

(before we get started)

W5 Announcements

Claude Subscription Support

- The university has generously provided us financial support - every enrolled student in this class receive **three months of Claude Team Subscription**
- The subscription goes through **Claude Team**, which requires everyone using your institution email (.edu)
 - Please create a Claude Account with you **institution email (.edu)**
 - **Fill in the Google Form** to provide us your registered account and **click this link to apply to join the Class's Claude team:**
 - We will verify your identity and approve after receiving your form response

W5 Announcements

Course Start Survey – Thank you!

- Thank you all for filling in the course start survey. We really appreciate it! 🙏
- We have make changes according to the course start survey...

W5 Announcements

Classroom change

- We have received a good amount of feedback on the classroom's limitation and concerns (seat & desk size, wifi  & air condition  quality), so we've decided to move to a different site of lecture
- Nonetheless, there're still things that is unresolved (e.g., limited charging port)

W5 Announcements

Course Format from Week 6-7

- Week 6
 - Recorded Video (預錄影片)
 - TAs will be here on Monday, 11:00 AM, if you need any help with the lab
- Week 7
 - Lecture offered remotely (遠端同步上課)
 - TAs will be here on Monday, 11:00 AM to help with lab

W5 Announcements

Discord is UP

- By the beginning of class, 53 of you have joined the Discord server
 - If you haven't, please join it as soon as possible
- Please **change your nickname (暱稱) to your full name** so we know who you are when replying to you

W5 Announcements

Discord is UP

- The reason that we're offering Discord as an additional channel is for communication and exchange
 - **General announcements and discussions**
 - **Synchronous (同步内容)** : Lecture / Lab , quick chat and update when in class or asking questions about lecture content
 - **Asynchronous (非同步内容)** : Final Project / Assignment , discussion around assignments and final project
 - **Chat (閒聊)** : AI trends and news / Tools and resources , anything you'd like to discuss broadly about course-relevant topics (e.g., Jev, 6.1 Sol, new and trending techniques)

W5 Announcements

Other

- We have decided to move **Lab submission deadline** to **the end of Wednesday (23:59) each week** to leave time for people to finish them

Learning Objectives

- Describe and apply four core prompting techniques — zero-shot, few-shot, chain-of-thought, and self-correction — and explain when each is appropriate given the task, stakes, and cost
- Write a specification before generating and use it as an evaluation checklist
- Identify the five prompt structure components (role, context, task, format, constraints) and explain what each controls
- Evaluate AI output against specific criteria using LLM-as-judge principles

The guiding question for today

- *"Which prompting technique should I use for this task, and why?"*
- We're NOT seeking which technique produces the best output, but which technique is appropriate given the task, the stakes, and the cost?"
- We rely on *engineering judgment*, not trial and error

The running example for today

- All techniques demonstrated on the same task:

"Given a Python function with no docstring, write a docstring that accurately describes what it does, its parameters, its return value, and one common pitfall."

- A Python function is provided — the same function for all techniques
- Why code generation? Because code either works or it does not — technique differences are immediately visible

What today's lecture/lab covers

- First Part: Core techniques for prompt engineering
- Second Part: Structure, specification, and evaluation
- Lab (60 minutes): lab practice of prompting techniques

Why prompting techniques exist

- Now you have a capable model, and you write an instruction. Sometimes it works, but sometimes it **does not**.
- Recap: *maybe try different wording until something works?*
 - When you cannot explain why it worked, **you cannot predict when it will fail or hand it to a teammate who can maintain it**
 - The model predicts the next token based on everything before it — how you say something changes what the model predicts
- Techniques are methods for shaping that probability distribution toward the output you actually want

The question behind every technique choice

- Every technique adds **control** and **cost** at the same time
 - Technique cost barely matters if it is a one-time generation of a security-critical document
 - Technique cost matters enormously when it is a batch job generating 10,000 low-stakes outputs
 - Moreover, the same task in different contexts may warrant different techniques
- Here, we are introducing four common techniques, starting with one with **minimum** control

Technique 1: Zero-shot

The simplest possible approach

- Provide the instruction only, no examples, no reasoning scaffold

- Format:

You are a Python documentation expert.

I need you to write a docstring for this function:

```
def mystery(items):
```

```
    ...
```

- **When it works:** well-defined tasks with clear outputs that the model has seen many times in training
- **When it fails:** ambiguous specifications, complex format requirements, tasks requiring domain-specific conventions

Technique 1: Zero-shot — live demonstration

- **Observe:** what decisions did the model make without being asked?
 - *Did it choose a docstring format (Google, NumPy, reStructuredText)?*
 - *Did it identify the common pitfall without being told what to look for?*
 - *Did it describe the return value accurately?*
- As we've mentioned before, **if you do not specify**, then these are implicit decisions you delegated to the model.

Technique 2: Few-shot

- Provide examples of (input, output) pairs before your task
- Why it works: examples constrain the output distribution more reliably than instructions alone — the model learns the format from demonstration, not description
- Format: 2–3 examples then the actual task
- Key insight: the examples are doing two things simultaneously
 - Demonstrating the exact output format you want
 - Showing the reasoning style and level of detail you expect

Technique 2: Few-shot

What makes a good example

- **Representative:** examples that cover the typical case, not the edge case
- **Consistent:** all examples follow the exact same format
- **Appropriately complex:** similar complexity to the actual task — too simple and the model does not transfer the pattern
- **Unambiguous:** if the example docstring is itself poorly written, the model learns from that
-  **Common mistake:**
 - Choosing examples you **personally find interesting** rather than examples that demonstrate the target format

Technique 2: Few-shot

Live demonstration

- Compare with zero-shot output: what changed?
 - Did the format become more consistent?
 - Did the level of detail shift?
 - Did the pitfall identification improve?
- What did not change — and why?

Technique 3: Chain-of-thought

- Ask the model to reason step by step before producing the final answer
- **Implicit CoT:** "Think step by step before writing the docstring"
- **Explicit CoT:** "First describe what the function does. Then identify the parameters. Then identify the return value. Then identify one pitfall. Then write the docstring."
- **Why it works:** producing reasoning tokens makes correct answer tokens more probable — the model is conditionally predicting based on its own reasoning
- **When it helps most:** tasks requiring multi-step reasoning, tasks where the model needs to inspect the code carefully before describing it

Technique 3: Chain-of-thought

Live demonstration

- The model produces intermediate observations before the docstring
- Token cost changes — CoT outputs are longer, so you are paying for the reasoning steps
- Compare with the previous ones: does the docstring improve when the model reasons first?
- Is the quality improvement worth the extra latency and cost?

Technique comparison so far

- The pattern: each technique adds control at the cost of tokens and complexity
 - **Zero-shot:** most commonly used, fast, cheap, works for well-defined tasks
 - **Few-shot:** more consistent format, higher token cost, requires good examples
 - **Chain-of-thought:** better reasoning on complex tasks, highest token cost, most latency
- The engineering question that we should ask:
 - What level of control does this task require?

Technique 4: Self-correction

- Generate an initial output, then prompt the model to critique and revise it
- Structure: generate → critique → revise
- The critique step must be specific — not "is this good?" but:
 - "Does this docstring accurately describe what the function does?"
 - "Does it cover all parameters?"
 - "Does it identify a genuine pitfall, not a trivial one?"
- **Why specific critique matters:** a vague critique produces sycophantic approval — the model says "yes, this looks good" and changes nothing

Technique 4: Self-correction

The sycophancy trap

- **Vague critique:** "Review this docstring and improve it if needed"
 - **Model response:** "This docstring looks comprehensive and well-written. No changes needed."
- **Specific critique:** "Does this docstring identify a pitfall that a junior engineer would not know to avoid? If not, revise it to add one."
 - **Model response:** [actually identifies and adds a pitfall]
- Therefore, the key is that the critique must give the model something specific to check — not permission to approve

Technique 4: Self-correction

Live demonstration

- Compare all three: initial, critique, revised
- Notice: did the critique identify a real problem? Did the revision address it?

The four techniques

Technique	API calls	Relative cost	Best for
Zero-shot	1	1×	Well-defined tasks, high volume
Few-shot	1	1.5–3×	Format consistency matters
Chain-of-thought	1	1.5–2×	Multi-step reasoning required
Self-correction	2–3	2–4×	High-stakes, specific quality criteria

- The engineering question: what is the cost of a wrong output?
- NOT which technique is best, but **which is appropriate for this task and these stakes**

The four techniques: when to use each

- **Zero-shot:** well-defined tasks, known format, model has strong priors — fast and cheap
- **Few-shot:** format consistency matters, you have good examples, cost is acceptable
- **Chain-of-thought:** task requires careful reasoning, multi-step inference, accuracy over speed
- **Self-correction:** high-stakes output, specific quality criteria, willing to pay 2–3x the token cost
- Note: **These techniques are not mutually exclusive.** We could use few-shot + CoT, few-shot + self-correction — combinations are common in production

Note on technique sensitivity

- The same technique with slightly different wording can produce significantly different outputs
 - **Demo:** same few-shot prompt, first word of instruction changed
- "Write a docstring for this function" vs "Generate documentation for this function"
- Recall from Week 1.....
 - This is not a bug, it is the **probability distribution shifting**
- Engineering implication: test your prompts on multiple inputs before trusting them
- Next week we will explain why this happens and how to diagnose it

What we will do in the lab

- Same task, same function, four techniques: zero-shot, few-shot, CoT, self-correction
 - For each technique: prompt verbatim, full model output, 2–3 sentences of analysis
- Submission for lab today (**in pairs**): which technique would you trust in production for this task, and why?
- This lab is direct preparation for Assignment 1

Step-back prompting

- There is a fifth technique you will use in Assignment 1 — we will not practice it in today's lab, but it is worth knowing.
 - "Before writing the docstring, answer: what makes a good docstring for a recursive function?"
 - The model grounds the specific task in general principles first
- **When it helps:** when the model is making systematic errors that suggest it is applying the wrong general approach

Next: engineering beyond prompting

- Why technique choice alone is not enough
 - **Structure (Prompt structure)**
 - What goes where, and why it matters
 - **Specification (Specify before you generate)**
 - **Evaluation (LLM-as-judge)**
 - Using the model to evaluate its own output

The hidden structure problem

- Two engineers write prompts using the same technique
- One consistently gets reliable outputs. The other does not.
- The difference is almost never the technique — it is the structure
- A prompt has **components**. Understanding what each does lets you control the output more precisely.

Prompt structure: the five components

Role, context, task, format, constraints

- **Role:** who the model is in this interaction
 - "You are a Python documentation expert writing docstrings for a junior engineering audience"
- **Context:** background information the model needs to do the task well
 - "This function is part of a sorting library used in production. Clarity is more important than brevity."
- **Task:** what you want the model to do — as specific as possible
- **Format:** exactly what the output should look like
- **Constraints:** what the output must not do or must not contain

Role

- "You are a helpful assistant" — adds almost nothing
- "You are a Python documentation expert" — adds domain framing
- "You are a Python documentation expert writing for engineers who have never used this library before" — adds audience and purpose
- The role component is most useful when the task requires a specific expertise or perspective
- **Common mistake:** using the role to add flattery ("you are an expert") instead of useful context ("you are an expert writing for X audience")

Context

What the model needs to know that you already know

- Context is background information that changes how the model should interpret the task
- The model produces output calibrated to your actual situation, not the generic case
- Without it, the model makes assumptions about your situation (which is often wrong)

Context

What the model needs to know that you already know

- What to provide:
 - The environment: "This function is part of a production sorting library"
 - The audience: "written for junior engineers who have never used this library before"
 - Constraints outside the prompt: "Clarity matters more than brevity"
 - Conventions the model cannot infer: "Our team uses Google docstring format"

Delimiters and XML tags

How to incorporate your code

- When your prompt contains the code to document, you need to separate instructions from content
- Without delimiters: the model may treat part of the code as an instruction
- Claude responds especially well to XML tag structure — it is trained on this format

Example: with XML tags

Write a docstring for the function below.

```
<function>  
def mystery(items):  
    ...  
</function>
```

Cover: what it does, parameters, return value, one pitfall.

Task

Structure
Specification
Evaluation

The most important component, and the easiest to get wrong

- The task is what you want the model to do
 - it sounds simple, but vague tasks are the most common source of prompt failure
- Three failure patterns:
 - **Too broad:** "Explain this function" (explain to whom? At what depth? For what purpose?)
 - **Implicit assumptions:** "Improve this docstring" (improve by what measure? Clarity? Completeness? Style?)
 - **Multiple tasks in one:** "Write a docstring and explain the algorithm and suggest improvements" (the model has to prioritise; it will not do all three equally well)

Task

Structure
Specification
Evaluation

The most important component, and the easiest to get wrong

- The fix: **one task per prompt**, stated as a **specific action** with a **measurable outcome**
 -  "Explain this function"
 -  "Write a one-sentence description of what this function does, suitable for a junior engineer who has never seen recursion"

Format: be specific or get surprised

- **Unspecified format:** the model will chooses. It may be Google style, NumPy style, reStructuredText, or something else
- **Specified format:** "Use Google docstring format with Args, Returns, and Raises sections"
- **More specified:** [provide the exact template]
- For code generation tasks: structured output (JSON, specific format) almost always requires explicit format specification

Constraints: drawing the boundaries

What not to do is as important as what to do

- "Do not include example usage in the docstring"
- "The pitfall must be non-obvious. Do not describe something every Python developer already knows"
- "Keep the description under two sentences"
- Why constraints matter: models tend toward completeness and verbose output unless constrained
- Common mistake: stating constraints as preferences ("try to keep it brief") rather than requirements ("keep it under 50 words")

Specification

Specify before you generate

- Before prompting: write a specification of what you want
 - Specification for the docstring task:
 - **Function:** describe what it does in one sentence
 - **Parameters:** name, type, and purpose for each
 - **Returns:** type and what it represents
 - **Pitfall:** one non-obvious issue a junior engineer would not anticipate
 - **Format:** Google docstring style, 100 words maximum
- The specification becomes the evaluation checklist: does the output satisfy each criterion?

Why specification enables evaluation

- **Without specification:**
 - "I know it when I see it" — unreliable, inconsistent, unteachable
- **With specification:**
 - five criteria, each checkable, each either met or not
- This is the same discipline as writing a test before writing the code (TDD)
- In your final project...
 - Your specification is what the code steward checks during PR review

LLM-as-judge: systematic evaluation

- Use cases:
 - Evaluating a large batch of outputs where human review is not feasible
 - Automated regression testing: does a new version of a prompt still meet quality criteria?
 - Filtering outputs before they reach users: flag low-quality outputs for human review
- The judge and the generator should use different prompts — and ideally different system prompts or temperatures

LLM-as-judge

Designing the judge prompt

- The judge prompt needs:
 - Clear, binary criteria — each criterion is either met or not
 - Explicit instruction to find problems — "assume at least one criterion is not fully met"
 - Output format that makes the judgment actionable — "for each criterion: pass/fail and one sentence explaining why"
 - Criteria ordered from most to least important — the judge should fail on the most important criterion first

LLM-as-judge: the blind spots

- AI is reliable for code following established patterns — common algorithms, standard library usage, conventional structure
- AI is unreliable for:
 - Novel algorithms it has not seen patterns of in training
 - Security edge cases requiring domain-specific knowledge
 - Component interactions it cannot see from a single function
 - Performance characteristics that only emerge at scale
- When using LLM-as-judge on code: these are the cases where the judge may miss what the expert reviewer would catch
- Week 12 develops this fully — today is a preview

LLM-as-judge

Using the model to evaluate its own output

- Generate output → use a second prompt to evaluate it against specific criteria
- Why this works: evaluating output is a different task from generating output — different probability distributions are in play

Evaluate this output. For each criterion: PASS or FAIL and one sentence explaining why.

Criterion 1 (most important): [...]

Criterion 2: [...]

<output> ... </output>

- Blind spots: novel algorithms, security edge cases, component interactions, performance at scale
- Week 11 develops this fully — today is a preview

Cost and latency tradeoffs

- Zero-shot: 1 API call, baseline cost and latency
- Few-shot: 1 API call + example tokens — 2–4x the input cost
- Chain-of-thought: 1 API call, longer output — higher output cost
- Self-correction: 2–3 API calls — 2–3x total cost
- LLM-as-judge: 2 API calls — 2x total cost
- Production framework: use Haiku for test runs, Sonnet for production quality
- The engineering question: what is the cost of a wrong output? Is extra technique worth the extra cost?

A structured approach to prompt design

Let's put it all together

- Step 1: write a specification — what must the output contain and what format must it take?
- Step 2: start with zero-shot — does it meet the specification?
- Step 3: if not, diagnose — is it a format problem (try few-shot), a reasoning problem (try CoT), or a quality problem (try self-correction)?
- Step 4: test on multiple inputs — does the technique work consistently or only on the example you tested?
- Step 5: evaluate with LLM-as-judge — does programmatic evaluation catch the failures that matter?

Assignment 1 preview

- Part 1: five techniques on a series of programming task
 - Zero-shot, few-shot, chain-of-thought, step-back, self-correction
 - For each: prompt verbatim, full output, 2–3 sentence analysis
- Part 2: diagnose and rewrite three provided broken prompts
 - Failure mode (specific), mechanism (using course vocabulary), rewrite, evidence (both outputs), personal connection
- Graded on: technique correctly implemented, prompts meaningfully different, analysis specific to this task

Recap

- The four core techniques are tools. Technique choice is an engineering judgment.
- "Specify before you generate" is the most transferable habit in the course
- LLM-as-judge adds programmatic quality control — but has the same blind spots as the model
- Structure (role, context, task, format, constraints) is what makes technique knowledge reliable
- Week 6 (async): why prompts fail — the mechanisms behind what you saw today

Lab 5: Lab: Writing Prompts Well

Today's lab

- **Goal:** apply the four techniques to the same function, and judge each output against a specification you write first
 - Work individually until minute 40, then pair up
- **Materials:** lab guide, the lab function, and the few-shot examples are on the Course Website
- **Submit:** one document per pair on E3 by Wednesday, October 7, 23:59 (details on the last lab slide)

Today's lab

Minutes	What to do
0–5	Step 0: write your specification
5–12	Technique 1: zero-shot
12–20	Technique 2: few-shot
20–28	Technique 3: chain-of-thought
28–40	Technique 4: self-correction
40–50	Pair discussion + one shared exit-ticket paragraph
50–60	Buffer: assemble your pair submission

Today's lab

The function that we're working on

- `process_records` (on the course website)

```
def process_records(records, threshold):  
    result = []  
    seen = {}  
    for r in records:  
        key = r.get('id')  
        val = r.get('score', 0)  
        if key in seen:  
            continue  
        seen[key] = True  
        if val >= threshold:  
            result.append({'id': key, 'score': val, 'status': 'pass'})  
        else:  
            result.append({'id': key, 'score': val, 'status': 'fail'})  
    return result
```

- This is a **different function** from the one in the lecture demos
- Please read it first — understand what it does, what the parameters mean, what it returns, and how it might surprise a new engineer.
- The same function is used for **all four techniques**

Today's lab

- **Your task:** get the model to write a docstring for process_records that meets your specification. You write the prompts, the model writes the docstring, and you judge it
- **You produce:**
 - **Four docstrings** — one per technique, each judged against your own specification. At the end, your pair picks one to approve

Step 0: write your specification first

- Before you write any prompt, write down what a good *docstring* for process_records must contain
- Five fields:
 - What it does: one sentence
 - Parameters: name, type, purpose
 - Returns: structure, and what each field means
 - Edge cases: at least two behaviours a new engineer might not expect
 - Format: docstring style (e.g., Google) and a length limit
- Keep it visible. Every output today is checked against it, field by field

Step 1: Try the prompts

The four techniques: what to do

	Technique	What to write	What to look for
1	Zero-shot	A plain instruction. No examples, no reasoning scaffold	Which decisions did the model make for you?
2	Few-shot	The three provided examples, then the task	Did format and edge-case coverage improve?
3	Chain-of-thought	Ask the model to work through the function step by step (what it does → parameters → return value → an edge case), then write the docstring	Did the reasoning surface the edge cases from your specification?
4	Self-correction	Generate → critique → revise	Did the critique find a real problem, or approve everything?

What to record for each technique

Copy this template for each technique:

Technique N: [name]

Prompt (verbatim):

[paste the exact prompt, including the examples for few-shot]

Full output:

[paste the complete model output]

Docstring produced:

[paste the docstring itself. For chain-of-thought, the docstring after the reasoning steps; for self-correction, the revised version. For zero-shot and few-shot, write "same as full output".]

Analysis (2–3 sentences):

[Which of your specification fields did this meet? Which did it miss or get wrong? Name the field.]

- **For Technique 4:** record both prompts (generation and critique), the initial output, the critique output, and the revised output, then one analysis

Step 2: Pair discussion (minutes 40–50)

- **Pair up with the person next to you and compare your four outputs (5 min)**
 - Discuss: if you were the code steward reviewing a PR that adds this docstring, which of your docstrings would you approve, and which would you send back?
- **Pick one docstring your pair would approve (from either partner's four)**
 - You may edit it; note what you changed
- **Then write one shared paragraph (4–6 sentences) together explaining your choice (5 min). It must cover three things:**
 - Accuracy against your specifications
 - Format consistency
 - Cost, if this ran on thousands of functions
- **There is no single right answer. Your reasoning is what matters**

What to submit

- Only one partner needs to upload -> E3 (Week 5 Lab)
 - Deadline: Wednesday, October 7, 23:59 Format: one document per pair (PDF, Word, or plain text), or paste into the E3 text box
- ☐ Both names and student IDs, at the top of the document
- ☐ Each partner's specification: the five Step 0 fields, labelled with the partner's name
- ☐ Each partner's four technique records: that partner's own prompt, full output, docstring produced, and 2–3 sentence analysis
- ☐ Your pair's approved docstring (edits noted) and one shared paragraph explaining the choice: 4–6 sentences

What to submit

- We encourage detailed analyses that cite your specification and the actual output.
- The more specific you are, the more useful this lab will be for Assignment 1.

Learning Objectives

- Describe and apply four core prompting techniques — zero-shot, few-shot, chain-of-thought, and self-correction — and explain when each is appropriate given the task, stakes, and cost
- Write a specification before generating and use it as an evaluation checklist
- Identify the five prompt structure components (role, context, task, format, constraints) and explain what each controls
- Evaluate AI output against specific criteria using LLM-as-judge principles

Todos for this week

- *Week 5 Self-paced Module + Response*: APE and self-consistency (~60 min). Remember to complete it by the end of this Sunday
- Remember to submit your *Lab 5* by the end of Wednesday
- Keep an eye on *assignment 1 (Due: 10/30) info release* **by the end of Friday**
- Keep an eye on *Claude Team Subscription information* **by the end of Friday**
- Join *Class Discord Server* if you haven't

Week 6 Preview

- Week 6 is async — both instructors traveling. No in-person class.
- Async video lecture posted by Monday morning (~35 min)
- Async lab: hallucination red-team notebook (on E3)
- CP1 will be released by the end of this week
 - due by end of week 8 October 12th — submitted to E3
- TA drop-in session for Week 6: time will be posted on the course website

CSIC30216

AI-in-the-Loop in Software Project Cycle

AI 導入軟體專案流程

Week 5:
Prompt Engineering I -
Writing Prompts Well

Instructors:
Dennis Wang 王邦任
Yu-Chun (Grace) Yen 顏羽君

Date: 2026-10-05